

Spring AOP

Pourquoi la POA ?

Pourquoi des proxies en AOP ?

Concepts de base de la POA

Implémentation avec Spring AOP

1. Dépendance Maven (si aspectj)
2. Exemple de classe métier
3. Aspect de logging

Spring AOP fonctionne-t-il sur les méthodes privées ?

Spring AOP fonctionne-t-il si j'appelle une méthode depuis la même classe ?

Pourquoi Spring AOP ne marche pas quand tu appelles une méthode dans la même classe ?

Pourquoi la POA ?

En POO, certaines logiques **transversales** (comme la sécurité, les logs, les transactions) sont répétées partout → **code dupliqué, difficile à maintenir**.

Exemples de préoccupations transversales :

- Logs
- Transactions
- Sécurité
- Gestion des exceptions
- Temps d'exécution

AOP permet de séparer ces logiques dans des "aspects" dédiés, sans polluer le code métier.

Pourquoi des proxies en AOP ?

Spring AOP fonctionne en **interceptant l'appel d'une méthode, sans modifier le code source**.

Pour faire ça, Spring **crée un proxy** autour de ton bean :

- Le proxy **intercepte** les appels.
- Il **exécute ton aspect** (log, sécurité, etc).
- Puis il appelle la vraie méthode du bean.

```
1 Client → Proxy → Bean Réel
2           ↑
3       (Aspects : logs, sécurité, etc)
```

Concepts de base de la POA

Concept	Définition simple
Aspect	Classe qui regroupe une préoccupation transversale (ex: LoggingAspect).
Join Point	Point d'exécution possible (ex: méthode appelée).
Pointcut	une règle (expression) qui dit où l'aspect doit s'appliquer.
Advice	Code à exécuter à ce Join Point (avant, après, autour...).
Weaving	Processus qui injecte l'Advice au Join Point.

Types de Advice:

Advice type	Quand le code est exécuté
<code>@Before</code>	Avant l'exécution de la méthode
<code>@After</code>	Après l'exécution (succès ou erreur)
<code>@AfterReturning</code>	Après exécution réussie
<code>@AfterThrowing</code>	Si exception levée
<code>@Around</code>	Entoure toute l'exécution de la méthode (permet de la bloquer, modifier args...)

Implémentation avec Spring AOP

1. Dépendance Maven (si aspectj)

```

1 <dependency>
2   <groupId>org.springframework.boot</groupId>
3   <artifactId>spring-boot-starter-aop</artifactId>
4 </dependency>

```

2. Exemple de classe métier

```

1 @Service

```

```

2 public class UserService {
3     public void createUser(String name) {
4         System.out.println("User created: " + name);
5     }
6 }

```

3. Aspect de logging

```

1 @Aspect
2 @Component
3 public class LoggingAspect {
4
5     @Before("execution(* com.example.service.UserService.*(..))")
6     public void logBefore(JoinPoint joinPoint) {
7         System.out.println("Before: " +
8             joinPoint.getSignature().getName());
9     }
10
11     @AfterReturning("execution(* com.example.service.UserService.*
12         (..)")
13     public void logAfter() {
14         System.out.println("Méthode exécutée avec succès");
15     }
16 }

```

Ici, le **JoinPoint** est l'appel d'une méthode dans **UserService** (comme **getUser()**, **createUser()** ...).

Le paramètre **JoinPoint joinPoint** te donne **des infos sur ce point d'exécution** :

joinPoint.getSignature().getName() te donne le **nom de la méthode appelée** (ex: **"createUser"**).

```

1 @Aspect
2 @Component
3 public class PerformanceAspect {
4
5     @Around("execution(* com.example.service.UserService.*(..))")
6     public Object measureExecutionTime(ProceedingJoinPoint pjp) throws
7         Throwable {
8         long start = System.currentTimeMillis();
9
10        // Exécution de la méthode réelle
11        Object result = pjp.proceed();
12
13        long end = System.currentTimeMillis();
14        System.out.println("Temps d'exécution de " +
15            pjp.getSignature().getName() + ": " + (end - start) + " ms");
16
17        return result; // important : retourner le résultat
18    }
19 }

```

Spring AOP fonctionne-t-il sur les méthodes privées ?

Non, car Spring AOP passe par un proxy → il intercepte uniquement les **appels publics** via Spring.

Spring AOP fonctionne-t-il si j'appelle une méthode depuis la même classe ?

Non ! Car l'appel se fait **en interne**, sans passer par le proxy.

Solution : extraire la méthode dans un autre bean.

Pourquoi Spring AOP *ne marche pas* quand tu appelles une méthode dans la même classe ?

Parce que **Spring AOP utilise un proxy** (JDK proxy ou CGLIB).

Le proxy intercepte les appels *de l'extérieur* vers l'objet :

```
1 [Caller] → [Proxy] → [TonBean]
```

Mais si une méthode A appelle une méthode B **dans la même classe**, l'appel devient :

```
1 TonBean.methodA() → TonBean.methodB()
```

Le proxy n'est jamais appelé.

Donc **pas d'aspect**, pas de `@Transactional`, pas de `@Around`, etc.

C'est un appel **direct**, interne, naturel → **le proxy est contourné**.

Exemple concret

```
1 @Service
2 public class DemoService {
3
4     @Transactional
5     public void methodA() {
6         methodB(); // NO AOP, NO TRANSACTION !!!
7     }
8
9     @Transactional
10    public void methodB() {
11        // ...
12    }
13 }
14
```

`methodB()` **ne sera jamais transactionnelle** si elle est appelée depuis `methodA()` .

Le proxy ne peut intercepter que ceci :

```
1 demoService.methodB(); // appelé de l'extérieur → OK, aspect déclenché
```

Pas ceci :

```
1 this.methodB(); // appel interne → pas d'aspect
```

Mais attention : il existe des exceptions

Voici les trois cas où *ça marche quand même* :

Si tu forces l'appel à passer par le proxy

Exemple :

```
1 ((DemoService) AopContext.currentProxy()).methodB();  
2
```

Il faut activer :

```
1 @EnableAspectJAutoProxy(exposeProxy = true)  
2
```

→ Dans ce cas l'appel interne devient :

```
1 methodA → proxy → methodB  
2
```

aspect déclenché

Si tu utilises **AspectJ compile-time ou load-time weaving**

Ce n'est plus du proxy Spring.

C'est du vrai weaving → AOP bytecode-level.

Dans ce cas :

```
1 methodA → methodB
```

→ **Les deux sont interceptés**, même en interne.

Si tu mets la méthode dans un autre bean (la solution recommandée)

```
1 ServiceA.methodA() → ServiceB.methodB()  
2
```

Dans ce cas, Spring AOP fonctionne toujours.